Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЕНИЯ

Кафедра интеллектуальных информационных технологий

Отчет
Лабораторная работа №6

По дисциплине: Численные методы

Выполнил: Рассохов Е.П., гр. 421703

Проверил: Князева Л.П.

Минск, 2025

Численное решение задачи Коши для ОДУ первого порядка и их систем
Вариант: 8

Задание 1: Решить задачу Коши для дифференциального уравнения первого порядка на отрезке [0, 1]:

a) методом Эйлера-Коши с шагом h1=0,1 и h2=0,05 построить графики полученных решений;

b) методом Рунге-Кутта 4-го порядка с шагом h1=0,1 и h2=0,05, построить графики полученных решений;

c) с помощью функций DSolve и NDSolve, построить графики.

Сравнить все полученные решения. Сделать выводы о точности методов в

зависимости от шага сетки.

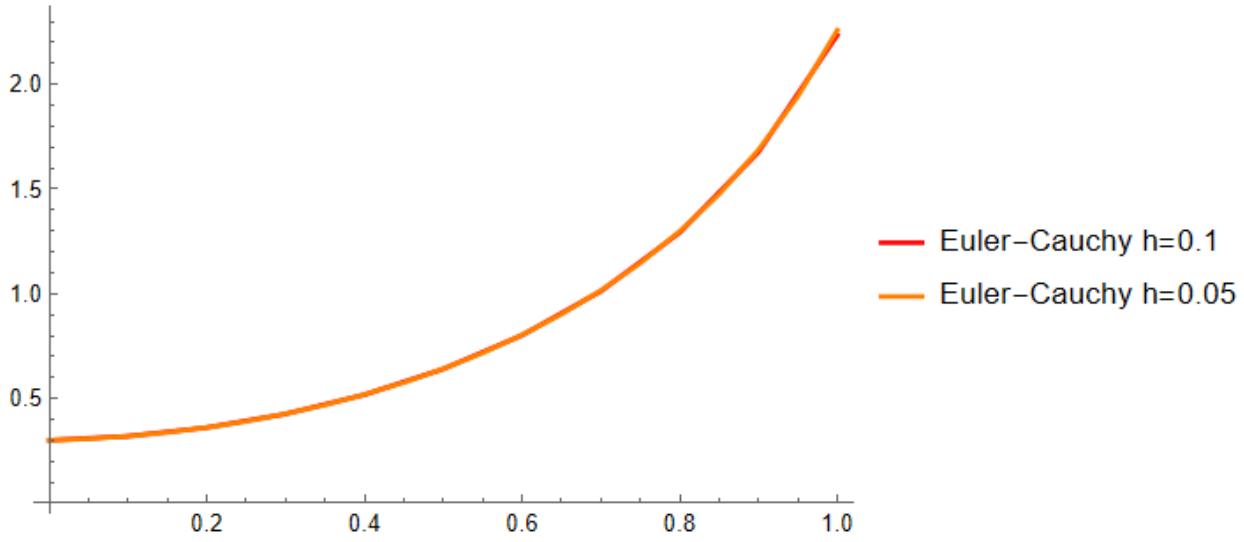
Подсчет отклонения, результата и опорных значений (Вместо NDSolve используется NDSolveValue для удобства подсчета отклонений, т. к. Она возвращает функцию)

```
exact = DSolve[{y'[x] == f[x, y[x]], y[0] == y0}, y, x];
        решить дифференциальные уравнения
exactNum = NDSolveValue[{y'[x] == f[x, y[x]], y[0] == y0}, y, {x, x0, xMax}];
           решение ДУ

ec1 = Euler[f, x0, y0, 0.1, xMax];
ec2 = Euler[f, x0, y0, 0.05, xMax];
r1 = Rung[f, x0, y0, 0.1, xMax];
r2 = Rung[f, x0, y0, 0.05, xMax];

Err[pts_, sol_] := N[Mean[Abs[pts[[All, 2]] - sol /@ pts[[All, 1]]]]];
                   ·· сре··· абсолю··· всё                  всё
```

a) Реализация метода Эйлера-Коши:

```
Euler[f_, x0_, y0_, h_, xMax_] := Module[
                                  программный модуль
   {pts = {{x0, y0}}, x = x0, y = y0, k1, yPred},
   While[x < xMax - 10^-12,
   цикл-пока
    k1 = f[x, y];
    yPred = y + h k1;
    y = y + (h / 2) (k1 + f[x + h, yPred]);
    x = x + h;
    AppendTo[pts, {x, y}];
    добавить в конец к
   ];
   pts];
```

График метода Эйлера-Коши:

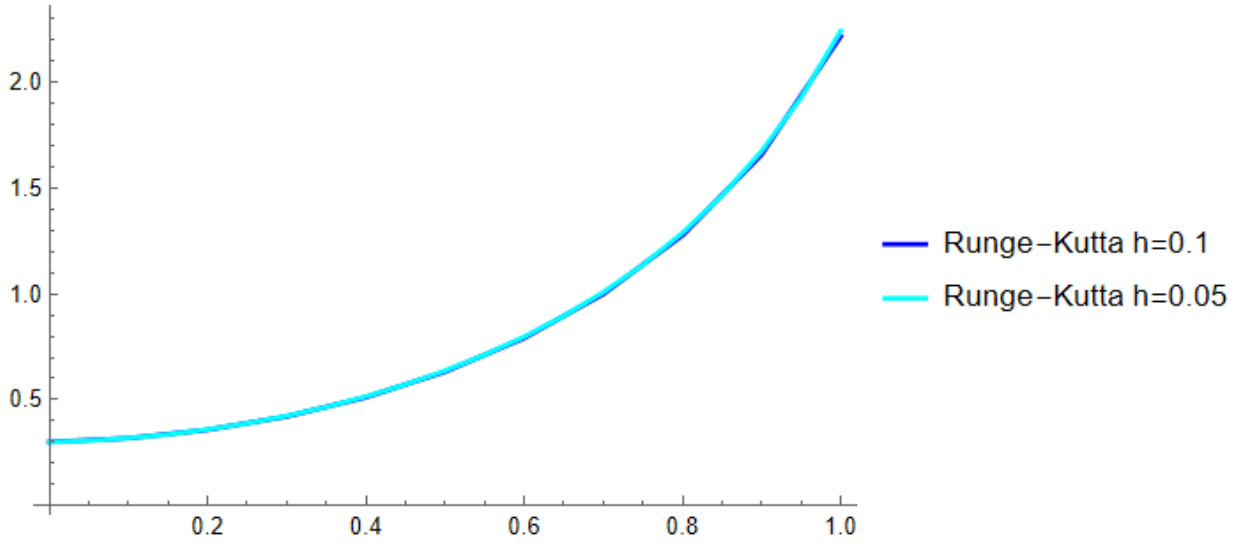



```
Error Euler-Cauchy h=0.1: 0.00605244
Error Euler-Cauchy h=0.05: 0.00139065
```

б) Реализация метода Рунге-Кутта 4-го порядка:

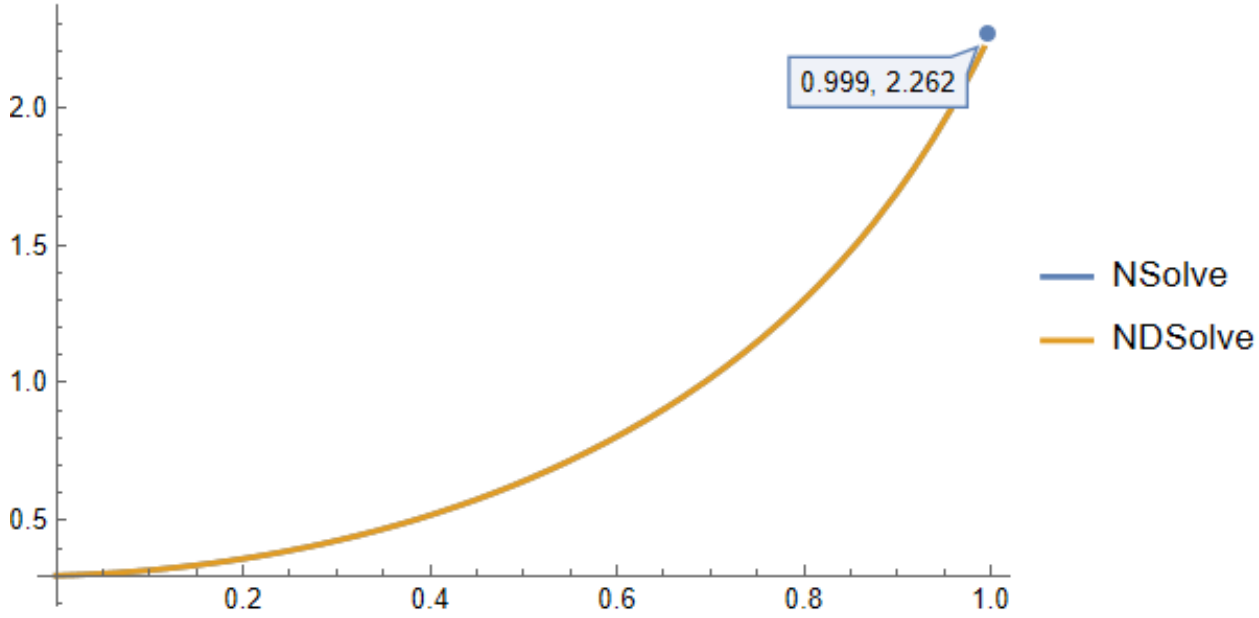
```
Rung[f_, x0_, y0_, h_, xMax_] := Module[
                                 программный модуль
  {pts = {{x0, y0}}, x = x0, y = y0, k1, k2, k3, k4},
  While[x < xMax - 10^-12,
  цикл-пока
   k1 = f[x, y];
   k2 = f[x + h / 2, y + h k1 / 2];
   k3 = f[x + h / 2, y + h k2 / 2];
   k4 = f[x + h / 2, y + h k3];
   y = y + (h / 6) (k1 + 2 k2 + 2 k3 + k4 );
   x = x + h;
   AppendTo[pts, {x, y}];
   добавить в конец к
  ];
  pts];
```

График метода Рунге-Кутта 4-го порядка:




```
Error Runge-Kutta h=0.1: 0.0153095
Error Runge-Kutta h=0.05: 0.00757142
```

в) Графики DSolve и NDSolve:



Вывод: Реализованы методы Эйлера–Коши и Рунге–Кутта четвёртого порядка, а эталонное решение получено через NDSolveValue. Сравнение показало, что метод Рунге–Кутта даёт заметную погрешность, которая уменьшается при уменьшении шага, но остаётся достаточно большой. Метод Эйлер–Коши, напротив, обеспечивает высокую точность даже при относительно крупном шаге, а при уменьшении шага точность становится

еще меньше. Все значения для подсчета среднего отклонения считались от вывода функции NDSolveValue

2. Решить задачу Коши для системы дифференциальных уравнений на отрезке [0, 1]:

a) методом Эйлера с шагом h1=0,1 и h2=0,05 построить графики полученных решений;

b) методом Рунге-Кутта 4-го порядка с шагом h1=0,1 и h2=0,05, построить графики полученных решений;

c) с помощью функций DSolve и NDSolve, построить графики.

Сравнить все полученные решения. Сделать выводы о точности методов в

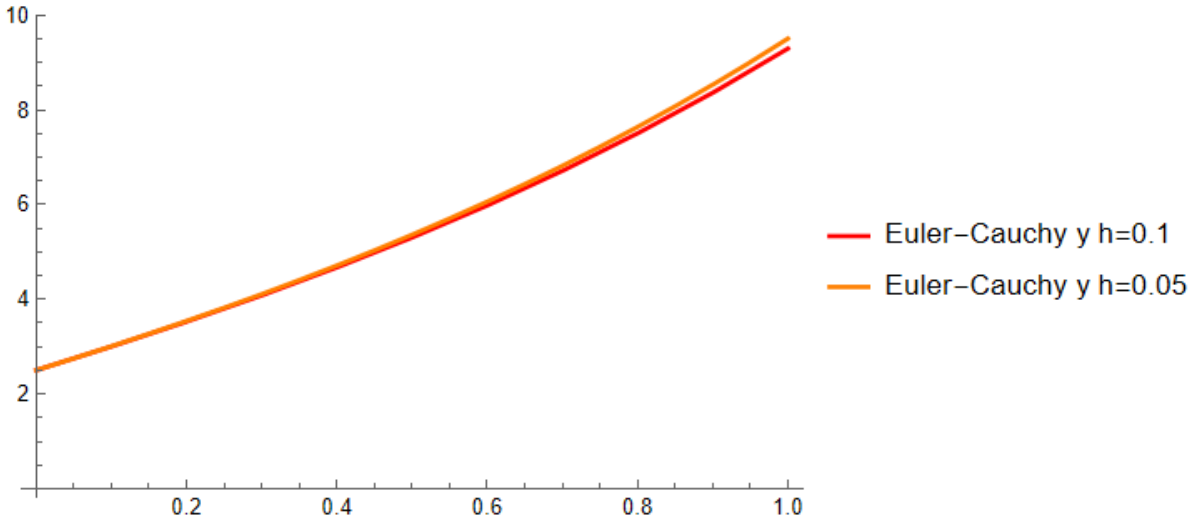
зависимости от шага сетки.

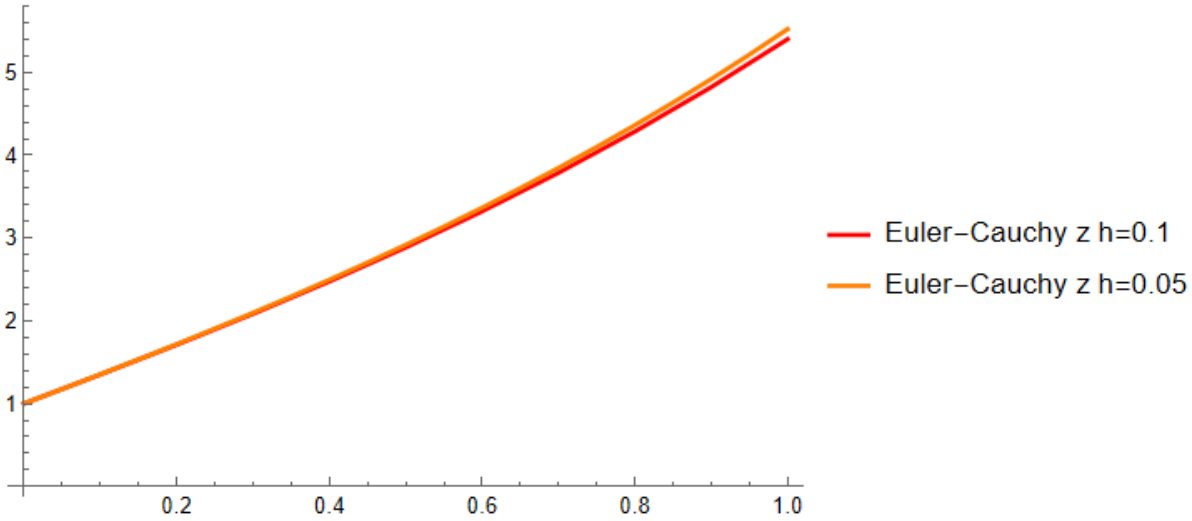
```
exact = DSolve[{y'[x] == 4 y[x] - 5 z[x], z'[x] == 3 y[x] - 4 z[x], y[0] == 2.5, z[0] == 1}, {y, z}, x];
        решить дифференциальные уравнения
exactNum = NDSolveValue[{y'[x] == 4 y[x] - 5 z[x], z'[x] == 3 y[x] - 4 z[x], y[0] == 2.5, z[0] == 1}, {y, z}, {x, 0, 1}];
           решение ДУ


e1 = Euler[F, 0, {2.5, 1}, 0.1, 1];
e2 = Euler[F, 0, {2.5, 1}, 0.05, 1];
r1 = Runge[F, 0, {2.5, 1}, 0.1, 1];
r2 = Runge[F, 0, {2.5, 1}, 0.05, 1];

Err[pts_, yf_, zf_] := Module[
                       программный модуль
   {xs, ys, zs},
   {xs, ys, zs} = Transpose[pts];
                  транспозиция
   {
    N[Mean[Abs[ys - yf /@ xs]]],
    ·· сре··· абсолютное значение
    N[Mean[Abs[zs - zf /@ xs]]]
    ·· сре··· абсолютное значение
   }];
```

a) Реализация метода Эйлера:

```
Euler[F_, x0_, {y0_, z0_}, h_, xMax] := Module[
                                        программный модуль
   {pts = {{x0, y0, z0}}, x = x0, y = y0, z = z0, dy, dz},
   While[x < xMax,
   цикл-пока
    {dy, dz} = F[{x, y, z}];
    y = y + h dy;
    z = z + h dz;
    x = x + h;
    AppendTo[pts, {x, y, z}];
    добавить в конец к
   ];
   pts];
```

Графики метода Эйлера:



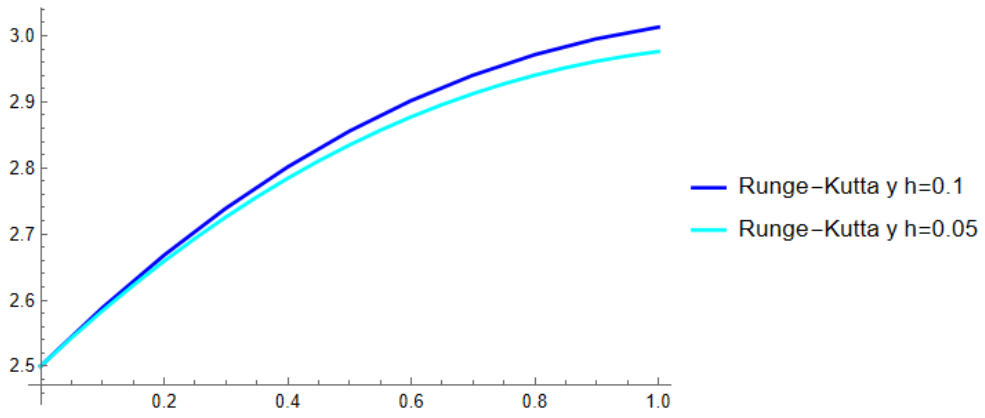


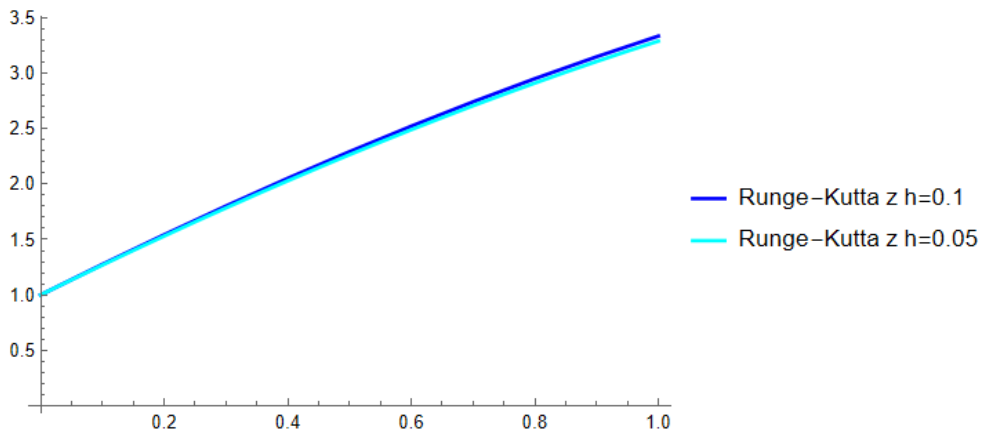

```
Error Euler-Cauchy h=0.1 (y, z): {0.162137, 0.0905314}
Error Euler-Cauchy  h=0.05 (y, z): {0.0832044, 0.0465842}
```

б) Реализация метода Рунге-Кутта 4-го порядка:

```
Runge[F_, x0_, {y0_, z0_}, h_, xMax] := Module[
                                        программный модуль
  {pts = {{x0, y0, z0}}, x = x0, y = y0, z = z0, k1, k2, k3, k4},
  While[x < xMax,
  цикл-пока
   k1 = F[{x, y, z}];
   k2 = F[x + h / 2, y + h k1[[1]] / 2, z + h k1[[2]] / 2];
   k3 = F[x + h / 2, y + h k2[[1]] / 2, z + h k2[[2]] / 2];
   k4 = F[x + h, y + h k3[[1]], z + h k3[[2]]];
   y = y + (h / 6) (k1[[1]] + 2 k2[[1]] + 2 k3[[1]] + k4[[1]]);
   z = z + (h / 6) (k1[[2]] + 2 k2[[2]] + 2 k3[[2]] + k4[[2]]);
   x = x + h;
   AppendTo[pts, {x, y, z}];
   добавить в конец к
  ];
  pts];
```

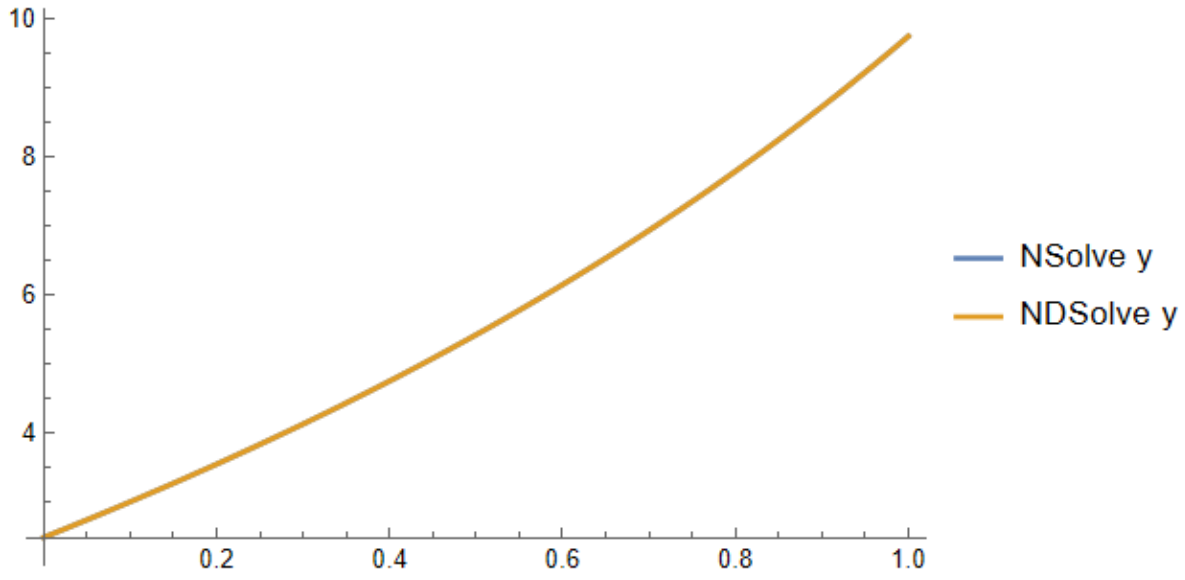
Графики метода Рунге-Кутта 4-го порядка:

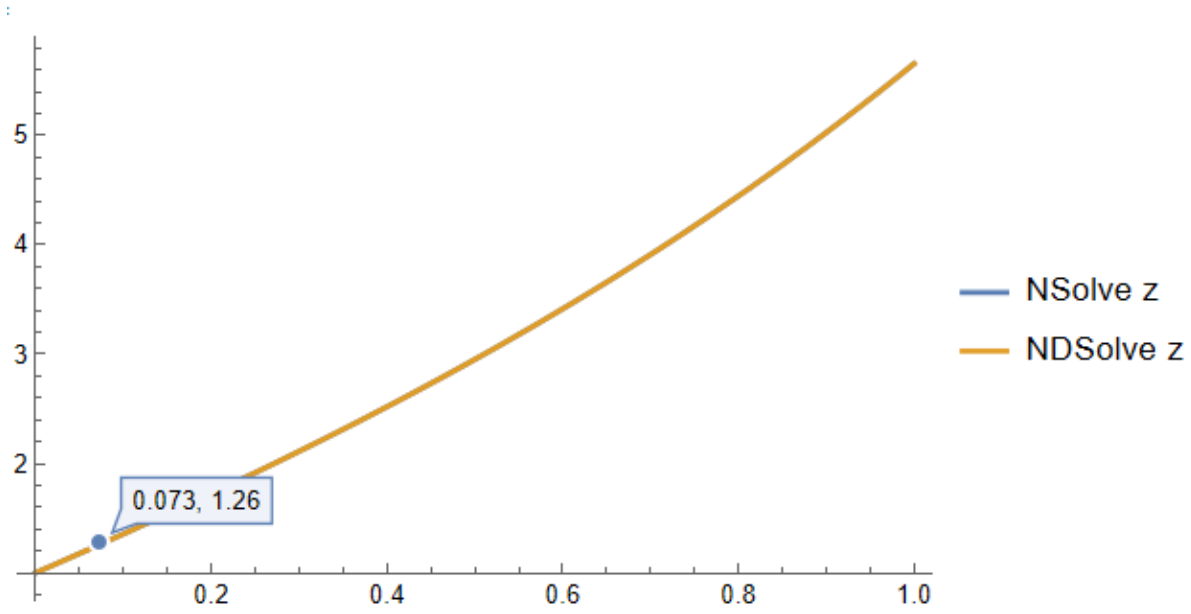





```
Error Runge-Kutta y h=0.1 (y, z): {2.88399, 0.860599}
Error Runge-Kutta y h=0.05 (y, z): {2.87752, 0.869316}
```

в) Графики DSolve и NDSolve:





Вывод: Здесь также было проведено сравнение методов Эйлера и Рунге–Кутта с эталонным решением. Результаты оказались аналогичными: метод Рунге–Кутта даёт ощутимую погрешность, которая уменьшается при уменьшении шага, но остаётся заметной. Метод Эйлера снова показал гораздо меньшую ошибку.